CS61C: Great Ideas in Computer Architecture (aka Machine Structures)

Lecture 5: C Wrapup, Floating Point

Instructor: Justin Yokota

Agenda

- Endianness
- Function Pointers
- Void Pointers
- Floating Point
  - Simplified Model
  - Optimizations
  - IEEE-754 Standard

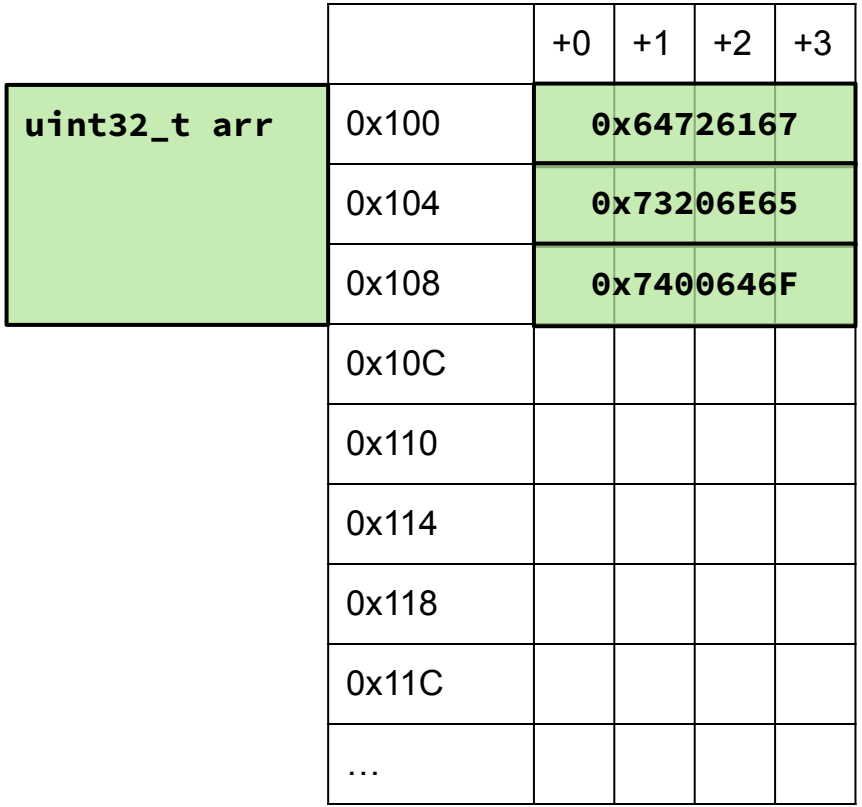
Agenda

- **Endianness**
- Function Pointers
- Void Pointers
- Floating Point
  - Simplified Model
  - Optimizations
  - IEEE-754 Standard

Endianness

What does the following code print?

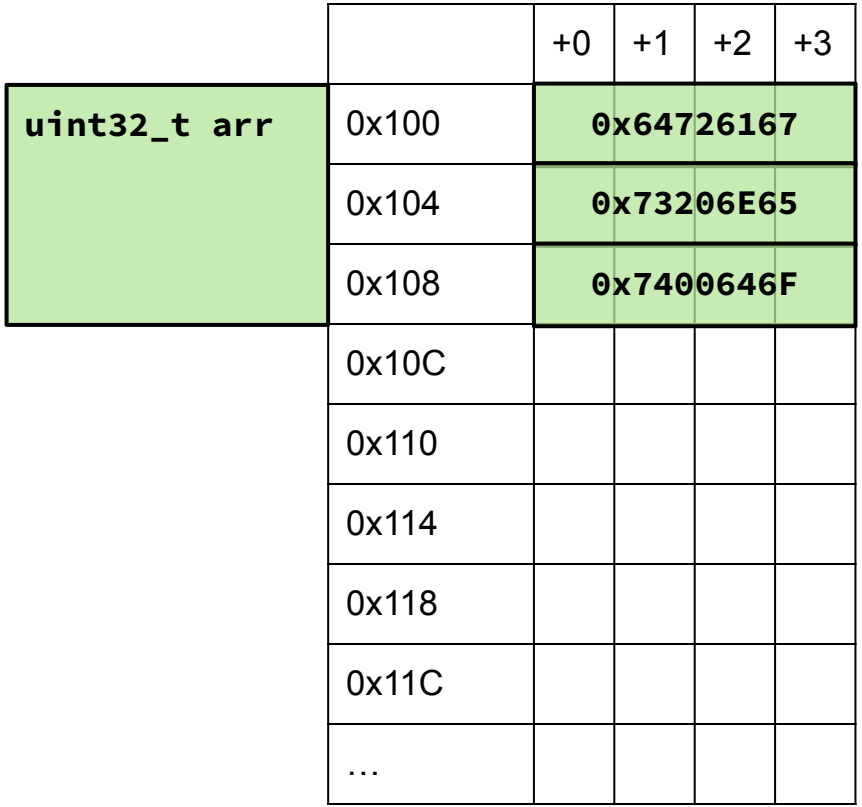
```
uint32_t arr[] =
{0x64726167,
0x73206E65,
0x7400646F};
printf("%x\n",arr[0]);
printf("%c\n",(char*)*arr);
printf("%s\n",arr);
```



Endianness

The first line should print 0x64726167

```
uint32_t arr[] =
{0x64726167,
0x73206E65,
0x7400646F};
printf("%x\n",arr[0]);
printf("%c\n",(char*)*arr);
printf("%s\n",arr);
```



Endianness

The second line reads one byte located at 0x100 as a char. The specific data at that byte depends on how we split arr[0]

```
uint32_t arr[] =
{0x64726167,
0x73206E65,
0x7400646F};
printf("%x\n",arr[0]);
printf("%c\n",*(char*)arr);
printf("%s\n",arr);
```

		+0	+1	+2	+3
uint32_t arr	0x100	**0x64726167**			
	0x104	**0x73206E65**			
	0x108	**0x7400646F**			
	0x10C				
	0x110				
	0x114				
	0x118				
	0x11C				
	…				

Big Endianness vs Little Endianness

Two main ways you can split an int into bytes:
Option 1 (Big-endian): Order the bytes such that the **most significant byte** (byte with the biggest effect) comes first
Option 2 (Little-endian): Order the bytes such that the **least significant byte** (byte with the smallest effect) comes first

`arr[0]`	+0	+1	+2	+3
0x100	**`0x64726167`**			
Big Endian	0x64	0x72	0x61	0x67
Little Endian	0x67	0x61	0x72	0x64

Big Endianness vs Little Endianness

Both approaches work, as long as you are consistent with how you read and write the data.

Analogy: How do you write dates?

- Europe: 26-6-2025 (little endian)
- China/Japan: 2025年6月26日 (big endian)
- US: 6/26/2025 (middle endian)

Mostly up to preference; everyone can tell the date regardless of how they write it

`arr[0]`	+0	+1	+2	+3
0x100	**0x64726167**			
Big Endian	0x64	0x72	0x61	0x67
Little Endian	0x67	0x61	0x72	0x64

Big Endianness vs Little Endianness

- Little-endian is common architectures (unless otherwise stated, assume little endian)
- Big-endian is common in network communications.

- Standardization is hard, okay?

`arr[0]`	+0	+1	+2	+3
0x100	**`0x64726167`**			
Big Endian	0x64	0x72	0x61	0x67
Little Endian	0x67	0x61	0x72	0x64

Big Endian vs Little Endian

Let's try running this same code on the two different endiannesses

```
uint32_t arr[] =
{0x64726167,
0x73206E65,
0x7400646F};
printf("%x\n",arr[0]);
printf("%c\n",(char*)*arr);
printf("%s\n",arr);
```

	Big-end	+0	+1	+2	+3
uint32_t arr	0x100	**0x64726167**			
	0x104	**0x73206E65**			
	0x108	**0x7400646F**			

	Little-end	+0	+1	+2	+3
uint32_t arr	0x100	**0x64726167**			
	0x104	**0x73206E65**			
	0x108	**0x7400646F**			

Big Endian vs Little Endian

When we create the array, we set data to appropriate values. The data gets split by the compiler according to the endianness of the system (0x omitted)

```
uint32_t arr[] =
{0x64726167,
0x73206E65,
0x7400646F};
printf("%x\n",arr[0]);
printf("%c\n",(char*)*arr);
printf("%s\n",arr);
```

	Big-end	+0	+1	+2	+3
uint32_t arr	0x100	64	72	61	67
	0x104	73	20	6E	65
	0x108	74	00	64	6F

(0x omitted in bytes)

	Little-end	+0	+1	+2	+3
uint32_t arr	0x100	67	61	72	64
	0x104	65	6E	20	73
	0x108	6F	64	00	74

Big Endian vs Little Endian

`arr[0]` is an int, consisting of the bytes 0x100, 0x101, 0x102, and 0x103. We read those 4 bytes, and combine them to 1 int.

```
uint32_t arr[] =
{0x64726167,
0x73206E65,
0x7400646F};
printf("%x\n",arr[0]);
printf("%c\n",(char*)*arr);
printf("%s\n",arr);
```

	Big-end	+0	+1	+2	+3
uint32_t arr	0x100	64	72	61	67
	0x104	73	20	6E	65
	0x108	74	00	64	6F

(0x omitted in bytes)

	Little-end	+0	+1	+2	+3
uint32_t arr	0x100	67	61	72	64
	0x104	65	6E	20	73
	0x108	6F	64	00	74

Big Endian vs Little Endian

Big endian: data is 64, 72, 61, 67

Combine the data with the first byte being most significant:

0x64726167

```
uint32_t arr[] =
{0x64726167,
0x73206E65,
0x7400646F};
printf("%x\n",arr[0]);
printf("%c\n",(char*)*arr);
printf("%s\n",arr);
```

	Big-end	+0	+1	+2	+3
uint32_t arr	0x100	64	72	61	67
	0x104	73	20	6E	65
	0x108	74	00	64	6F

(0x omitted in bytes)

	Little-end	+0	+1	+2	+3
uint32_t arr	0x100	67	61	72	64
	0x104	65	6E	20	73
	0x108	6F	64	00	74

Big Endian vs Little Endian

Little endian: data is 67, 61, 72, 64

Combine the data with the first byte being least significant:

0x64726167

```
uint32_t arr[] =
{0x64726167,
0x73206E65,
0x7400646F};
printf("%x\n",arr[0]);
printf("%c\n",(char*)*arr);
printf("%s\n",arr);
```

	Big-end	+0	+1	+2	+3
uint32_t arr	0x100	64	72	61	67
	0x104	73	20	6E	65
	0x108	74	00	64	6F

(0x omitted in bytes)

	Little-end	+0	+1	+2	+3
uint32_t arr	0x100	67	61	72	64
	0x104	65	6E	20	73
	0x108	6F	64	00	74

Big Endian vs Little Endian

Regardless of endianness, the int we write is the int we read. **Endianness only affects when we read a part of a block at a time.**

```
uint32_t arr[] =
{0x64726167,
0x73206E65,
0x7400646F};
printf("%x\n",arr[0]);
printf("%c\n",(char*)*arr);
printf("%s\n",arr);
```

	Big-end	+0	+1	+2	+3
uint32_t arr	0x100	64	72	61	67
	0x104	73	20	6E	65
	0x108	74	00	64	6F

(0x omitted in bytes)

	Little-end	+0	+1	+2	+3
uint32_t arr	0x100	67	61	72	64
	0x104	65	6E	20	73
	0x108	6F	64	00	74

Big Endian vs Little Endian

We read the byte at 0x100

For Big-endian: Byte is 0x64 = 'd'

For Little-endian: Byte is 0x67 = 'g'

```
uint32_t arr[] =
{0x64726167,
0x73206E65,
0x7400646F};
printf("%x\n",arr[0]);
printf("%c\n",(char*)*arr);
printf("%s\n",arr);
```

	Big-end	+0	+1	+2	+3
uint32_t arr	0x100	64	72	61	67
	0x104	73	20	6E	65
	0x108	74	00	64	6F

(0x omitted in bytes)

	Little-end	+0	+1	+2	+3
uint32_t arr	0x100	67	61	72	64
	0x104	65	6E	20	73
	0x108	6F	64	00	74

Big Endian vs Little Endian

Strings are char arrays, so we read bytes starting at 0x100, in increasing order, until we hit a 00 byte.

```
uint32_t arr[] =
{0x64726167,
0x73206E65,
0x7400646F};
printf("%x\n",arr[0]);
printf("%c\n",(char*)*arr);
printf("%s\n",arr);
```

	Big-end	+0	+1	+2	+3
uint32_t arr	0x100	64	72	61	67
	0x104	73	20	6E	65
	0x108	74	00	64	6F

(0x omitted in bytes)

	Little-end	+0	+1	+2	+3
uint32_t arr	0x100	67	61	72	64
	0x104	65	6E	20	73
	0x108	6F	64	00	74

Big Endian vs Little Endian

Big endian: Sequence goes 64, 72, 61, 67, 73, 20, 6E, 65, 74, 00

= "drags net"

```
uint32_t arr[] =
{0x64726167,
0x73206E65,
0x7400646F};
printf("%x\n",arr[0]);
printf("%c\n",(char*)*arr);
printf("%s\n",arr);
```

	Big-end	+0	+1	+2	+3
uint32_t arr	0x100	64	72	61	67
	0x104	73	20	6E	65
	0x108	74	00	64	6F

(0x omitted in bytes)

	Little-end	+0	+1	+2	+3
uint32_t arr	0x100	67	61	72	64
	0x104	65	6E	20	73
	0x108	6F	64	00	74

Big Endian vs Little Endian

Little endian: Sequence goes 67, 61, 72, 64, 65, 6E, 20, 73, 6F, 64, 00

= "garden sod"

```
uint32_t arr[] =
{0x64726167,
0x73206E65,
0x7400646F};
printf("%x\n",arr[0]);
printf("%c\n",(char*)*arr);
printf("%s\n",arr);
```

	Big-end	+0	+1	+2	+3
uint32_t arr	0x100	64	72	61	67
	0x104	73	20	6E	65
	0x108	74	00	64	6F

(0x omitted in bytes)

	Little-end	+0	+1	+2	+3
uint32_t arr	0x100	67	61	72	64
	0x104	65	6E	20	73
	0x108	6F	64	00	74

Endianness summary

Both endiannesses work within C; it's technically undefined behavior to split an int into chars like we did here.

Remember: writing and reading is always consistent

Common mistake students have is to "reverse order" when writing the bytes, but then forgetting to "reverse reading order" when reading bytes (especially in more complex cases).

`arr[0]`	+0	+1	+2	+3
0x100	**`0x64726167`**			
Big Endian	0x64	0x72	0x61	0x67
Little Endian	0x67	0x61	0x72	0x64

Agenda

- Endianness
- **Function Pointers**
- Void Pointers
- Floating Point
  - Simplified Model
  - Optimizations
  - IEEE-754 Standard

More Pointers

Pointers can point to more than just variables!

- Function Pointers:
  - `int *(*fp) (int, int)`
  - Point to functions
  - Lets us implement higher order functions
- Generic Pointers:
  - `void* p`
  - Points to *anything*
  - Lets us implement generic functions

Function Pointers

Function pointers are variables storing addresses of a function

- Declaration:
  - `int *(*fp) (int, int) = &foo;   // Function fp (int,int)->int*`
  - `int *(*fp) (int, int) = foo;    // Can only be omitted for function pointers`
- Usage:
  - `(*fp)(x, y)`
  - `fp(x, y)    // Can only be omitted for function pointers`
- Note: Convention
  - Omitting & and `(*...)` follows the “less is more” ideology
  - Leaving them in follows the “be clear” / “defensive programming” ideology

Example: map


```c
/* Mutates an array ARR of length N by
 * replacing each element of ARR with
 * the result of calling FP on that element. */
void map(int arr[], size_t n, int (*fp)(int)) {
    for (int i = 0; i < n; i++) {
        arr[i] = fp(arr[i]); // remember that arr[] is a pointer!
    }
}

int times2 (int x) { return  2 * x; }
int times10(int x) { return 10 * x; }
```

Example: Using `map`


```c
#include <stdio.h>
#define ARR_LEN 3
int main() {
    int arr[ARR_LEN] = {3, 1, 4};
    printf("arr: [%d, %d, %d]\n", arr[0], arr[1], arr[2]);
    map(arr, ARR_LEN, &times2);
    printf("arr: [%d, %d, %d]\n", arr[0], arr[1], arr[2]);
    map(arr, ARR_LEN, &times10);
    printf("arr: [%d, %d, %d]\n", arr[0], arr[1], arr[2]);
}
```

```
$ gcc main.c && ./a.out
arr: [3, 1, 4]
arr: [6, 2, 8]
arr: [60, 20, 80]
```

Agenda

- Endianness
- Function Pointers
- **Void Pointers**
- Floating Point
  - Simplified Model
  - Optimizations
  - IEEE-754 Standard

Why use generic functions?

- Writing general-purpose code
  - Reduces code rewriting, avoids having to make the same change in multiple places
  - Used throughout C libraries for things like:
    - Free arbitrary memory
    - Sort any array
- In general,
  - Generics should work for any argument type
  - Updates blocks of memory regardless of data types

The `void*` pointer

- Doesn't mean "pointer to nothing" (yes the name is misleading)
- Instead means "pointer to anything"
- Quirks:
  - You can **never, ever** dereference a `void*` pointer
  - You can't do pointer arithmetic with `void*` pointers
    - Why?
  - `void*` pointers can be safely and automatically converted to other pointer types

```
void* ptr1;
/* some code */
int* ptr2 = ptr1;
```

  - Note: `void*` pointers cannot be automatically cast to function pointers.

Danger: Dereferencing

Compare the two below blocks of code. What should happen?

```
1   void *ptr = …;
2   printf(“%p\n”, *ptr); // Note: %p means “pointer”
```

```
1   void **doubleptr = …;
2   printf(“%p\n”, *doubleptr);
```

Danger: Dereferencing

Compare the two below blocks of code. What should happen?

```
1   void *ptr = …;
2   printf(“%p\n”, *ptr); // Note: %p means “pointer”
```

```
main.c:6:17: warning: ISO C does not allow indirection on operand of type 'void *'
     [-Wvoid-ptr-dereference]
     printf("%p\n", *ptr);
                    ^~~~
main.c:6:17: error: argument type 'void' is incomplete
```

```
1   void **doubleptr = …;
2   printf(“%p\n”, *doubleptr);
```

- Perfectly fine!
  - `doubleptr` is… **a pointer to…** a pointer to anything
  - Also, we know `sizeof(void *)`! (It’s the same size as any other pointer)

Example: C heap library


```
$ man malloc

NAME
     calloc, free, malloc, realloc, reallocf, valloc, aligned_alloc - memory allocation

SYNOPSIS
     #include <stdlib.h>

     void *
     calloc(size_t count, size_t size);

     void
     free(void *ptr);

     void *
     malloc(size_t size);

     void *
     realloc(void *ptr, size_t size);

     ...
```

Generic Memory Functions

We have two main functions to work with memory:

- `void* memcpy(void *dest, void *src, size_t count);`
  - Copies `count` bytes from `src` to `dest`.
  - Undefined behavior if `src` and `dest` overlap (i.e. `(char *) src + count >= dest`)
  - Fast!!
- `void* memmove(void *dest, void *src, size_t count);`
  - Copies `count` bytes from `src` to `dest`.
  - No problems if `src` and `dest` overlap (i.e. `(char *) src + count >= dest`)
  - Slow!!
- Both
  - Return a copy of `dest`
  - Fail if either `src` or `dest` are invalid or `NULL`.

Swapping


```
1  void swap_ints(int *x, int *y) {
2      int tmp = *x;
3      *x = *y;
4      *y = tmp;
5  }
```

```
1  void swap_floats(float *x, float *y) {
2      float tmp = *x;
3      *x = *y;
4      *y = tmp;
5  }
```

```
1  void swap_strings(char **x, char **y) {
2      char *tmp = *x;
3      *x = *y;
4      *y = tmp;
5  }
```

Swapping


```
1  void swap(void *x, void *y) {
2      /* Save x in tmp
3       * Copy value in y to x
4       * Copy value in tmp to x        */
5  }
```

- Idea: reuse swapping code
- However…
  - How do we know the size of the arguments?
  - Let's add a new argument

Swapping


```
1  void swap(void *x, void *y, size_t nbytes) {
2      /* Save x in tmp
3       * Copy value in y to x
4       * Copy value in tmp to x        */
5  }
```

- Idea: reuse swapping code
- However…
  - How do we know the size of the arguments?
  - Let's add a new argument

Swapping — Full Code


```
1  void swap(void *x, void *y, size_t nbytes) {
2      char tmp[nbytes];          // Create space to hold x;
3      memcpy(tmp, x  , nbytes); // Save value of x in tmp
4      memcpy(x  , y  , nbytes); // Copy value in y to x
5      memcpy(y  , tmp, nbytes); // Copy value in tmp to y
6  }
```

- In C, `sizeof(char) == 1` by definition
  - i.e. a `char` is C's version of a `byte`
- Why can't we replace line 4 with `*x = *y`?

Swapping — Full Code


```
1  void swap(void *x, void *y, size_t nbytes) {
2      char tmp[nbytes];          // Create space to hold x;
3      memcpy(tmp, x  , nbytes); // Save value of x in tmp
4      memcpy(x  , y  , nbytes); // Copy value in y to x
5      memcpy(y  , tmp, nbytes); // Copy value in tmp to y
6  }
```

- In C, `sizeof(char) == 1` by definition
  - i.e. a `char` is C's version of a `byte`
- Why can't we replace line 4 with `*x = *y`?
  - Dereferencing `void*`!

Swapping — Demo


```
1   int main(int argc, char *argv[]) {
2       int a = 6;
3       int b = 1;
4       printf("a: %d | b: %d\n", a, b);
5       swap(&a, &b, sizeof a);
6       printf("a: %d | b: %d\n", a, b);
7   }
```

```
$ gcc main.c && ./a.out
a: 6 | b: 1
a: 1 | b: 6
```

Generic Functions on Arrays

How do we do pointer arithmetic on `void*` pointers? Cast first!

- Given:

```
void swap(void *x, void *y, size_t nbytes)
```

- Let's write the following function:

```
void swap_ends(void *arr, size_t nelems, size_t nbytes)
```

- What type should we use for pointer arithmetic?
  - Hint: what type is guaranteed to have size 1?

Swap Ends


```
1  void swap_ends(void *arr, size_t nelems, size_t nbytes) {
2      void *start = arr;
3      void *end   = (__a__) arr + __b__;
4      swap(start, end, nbytes);
5  }
```

- What type should we use for pointer arithmetic?
  - Hint: what type is guaranteed to have size 1?

Swap Ends


```
1  void swap_ends(void *arr, size_t nelems, size_t nbytes) {
2      void *start = arr;
3      void *end   = (char *) arr + __b__;
4      swap(start, end, nbytes);
5  }
```

- What type should we use for pointer arithmetic?
  - Hint: what type is guaranteed to have size 1?

Swap Ends


```
1  void swap_ends(void *arr, size_t nelems, size_t nbytes) {
2      void *start = arr;
3      void *end   = (char *) arr + __b__;
4      swap(start, end, nbytes);
5  }
```

- How should we do our pointer arithmetic?
  - Hint: Normally, we would do `&arr[nelems - 1]` to refer to the address of the last element
  - Hint 2: Remember that `arr[nelems - 1]` means `*arr + (nelems - 1)`
  - Hint 3: Pointer arithmetic in C automatically multiplies by the size of what is being pointed to, but we don't have that luxury in generic pointer arithmetic — you have to multiply by the size

Swap Ends — Full Code


```
1  void swap_ends(void *arr, size_t nelems, size_t nbytes) {
2      void *start = arr;
3      void *end   = (char *) arr + (nelems - 1) * nbytes;
4      swap(start, end, nbytes);
5  }
```

- How should we do our pointer arithmetic?
  - Hint: Normally, we would do `&arr[nelems - 1]` to refer to the address of the last element
  - Hint 2: Remember that `arr[nelems - 1]` means `*(arr + (nelems - 1))`
  - Hint 3: Pointer arithmetic in C automatically multiplies by the size of what is being pointed to, but we don't have that luxury in generic pointer arithmetic — you have to multiply by the size

Swap Ends — Test


```c
#define ARR_LEN 4
int main(int argc, char *argv[]) {
    int arr[ARR_LEN] = {1, 4, 3, 4};
    printf("arr: [%d, %d, %d, %d]\n", arr[0], arr[1], arr[2], arr[3]);
    swap_ends(arr, ARR_LEN, sizeof arr[0]);
    printf("arr: [%d, %d, %d, %d]\n", arr[0], arr[1], arr[2], arr[3]);
}
```

```
$ gcc main.c && ./a.out
arr: [1, 4, 3, 4]
arr: [4, 4, 3, 1]
```

Agenda

- Endianness
- Function Pointers
- Void Pointers
- **Floating Point**
  - **Simplified Model**
  - Optimizations
  - IEEE-754 Standard

Storing Data in Binary

A system to data as binary is known as a **representation scheme**.

Ideally, schemes should:

- Represent data that's relevant to whatever we're making
- Store things efficiently
  - Optimal memory use
  - Efficient operators
- Be intuitive for humans to understand
  - Can skip this one if absolutely necessary

Today's goal: Develop a representation scheme for decimal/fractional numbers

IEEE-754 Floating Point Numbers

- An encoding scheme to store arbitrary real numbers
- It starts off nice
  - Able to store relevant values
  - Relatively simple circuitry
  - Intuitive for humans
- Subsequent optimizations make add more relevant values and reduce memory inefficiency, but make things less intuitive
  - Denorms, Infinities, NaNs
- We'll be building various "versions" of the Floating Point standard today to build up to IEEE-754 standard, but you only need to know the final version.

Goals: What numbers do we want to store?

- IEEE-754 was designed as a “universal” number system, in order to encourage consistency (don’t want to deal with translating between different conventions)
- As such, we want to handle:
  - Really big numbers (ex. Avogadro’s number = $6.022*10^{23}$)
  - Really small numbers (ex. Planck’s constant = $6.626*10^{-34}$)
- Notable observation: When dealing with large numbers, we generally don’t care about small absolute differences
  - Ex. If you’re dealing with a 10 kg bag of rice, you don’t care about the extra weight of a few grains of rice
  - If you’re measuring the weight of atoms, you probably care a lot about the extra weight of a few grains of rice

Goals: What numbers do we want to store?

- Goal: Attempt to store numbers to a certain **relative precision**
  - If we can't store a number exactly, make sure that we can store a number that's accurate to within 0.001%
  - Allows us to store a much wider range of numbers, but most numbers get "rounded" to the nearest representable number
- Contrast: ints store every possible integer between min and max value
  - Precise computations as long as numbers stay integer, but range is limited
- No free lunch: Each operation in a floating-point scheme causes some slight error
  - Some algorithms are **numerically unstable**; floating point error gets magnified over multiple operations, and causes a wrong answer even if the algorithm is mathematically correct
  - More info: Math 128A

Analogous system: Scientific Notation

- Goal: Attempt to store numbers to a certain relative precision
  - If we can't store a number exactly, make sure that we can store a number that's accurate to within 0.001%
- Scientific notation already does this!
- Ex: $6.022*10^{23}$ stores Avogadro's number using:
  - 1 bit (+/-)
  - 4 decimal digits (6022) (Called the mantissa or the significand)
  - An exponent (23)
- 4 decimal digits means we get precision up to 10^{-3} (numbers we store are accurate to within 0.1%)
- If we set limits on the exponent to +/- 500, we can store very small AND very large numbers using 2 numbers and a sign bit

Floating Point System: V0

- To store a floating point number, we need to save:
  - Sign bit: + or -
  - Mantissa: Positive integer with no leading zeros
  - Exponent: Positive or negative integer
- Since we're working with binary, we'll use scientific notation with a base of 2
  - Ex. We'd save something like 0b1.011 * 2^5
    - Warning: In binary, numbers after the binary point are smaller powers of 2. So 0b1.011 = $2^0+2^{-2}+2^{-3}=1+0.25+0.125 = 1.375$
- How to store mantissa?
  - Mantissa is just a positive integer, so we can save it as an unsigned number
- How to store exponent?

How should we store the exponent?

How should we store a floating point number's exponent?

- Options:
  - Sign-Magnitude
  - Bias
  - Two's Complement
- How do various floating point operations affect the exponent? Which format will give us the best balance of properties?

Floating Point System V0: Exponent storage options

- Three major types of operators that are relevant to both floats and ints
  - Add/Subtract (Ex. $5*10^4+6*10^8$, $1.234*10^4-1.233*10^4$)
    - Addition affects exponent in complicated ways; often around the max of the two exponents, but can be off by 1.
    - Subtraction yields completely different exponents
  - Multiply/Divide (Ex. $5*10^4 \times 6*10^8$, $1.234*10^4/1.233*10^4$)
    - Affects exponent in complicated ways; often around the sum/difference of the two exponents, but can be different.
  - Comparisons (Ex. $5*10^4<6*10^8$, $1.234*10^4>=1.233*10^4$)
    - Compare the exponent first, and if the two are equal, compare the mantissa
    - Similar to how we compare integers (compare the top digit, if equal, compare the next digit)
- Conclusion: The only operation that "plays nice" with exponents is comparison. **Bias notation** works well with comparisons.

Floating Point System: V0

- To store a floating point number, we need to save:
  - Sign bit: + or -
  - Mantissa: Positive integer with no leading zeros
  - Exponent: Positive or negative integer
- Since we're working with binary, we'll use scientific notation with a base of 2
  - Ex. We'd save something like $0b1.011 * 2^5$
    - Note: In binary, keep in mind that numbers after the binary point are smaller powers of 2. So $1.011 = 2^0+2^{-2}+2^{-3}=1+0.25+0.125 = 1.375$
- How to store mantissa?
  - Mantissa is just a positive integer, so we can save it as an unsigned number
- How to store exponent?
  - Exponent gets stored with a biased encoding. Data is ordered sign-exponent-mantissa so that we can reuse the comparator circuit from sign-magnitude numbers.
  - Bias is often set to $-(2^{n-1}-1)$ for an n-bit exponent, to have about equal positive/negative exponents.

Floating Point System: V1

- A floating point number uses x bits for the exponent, and y bits for the mantissa (with x,y specified as parameters). For this slide, we'll use as an example a system with 3 exponent bits (bias of -3) and 4 mantissa bits

S XXX MMMM

- This represents the number $(-1)^{S} * 0bM.MMM * 2^{(0bXXX+(-3))}$

Floating Point V1: Example

- For this example, let's assume we're working with a system with 3 exponent bits (bias of -3) and 4 mantissa bits.
- Convert 2.75 to a V1 float
- Step 1: Write the number in binary
  - 0b10.11 = 0b1.011000… * 2^1
- Step 2: Determine Sign/Exponent/Mantissa
  - Sign: Positive → 0
  - Exponent: 1-(-3) = 4 → 0b100
  - Mantissa: 0b1011
- Step 3: Concatenate
  - 0b0 100 1011
  - 0x4B

Floating Point V1: Example

- For this example, let's assume we're working with a system with 8 exponent bits (bias of -127) and 23 mantissa bits.
- Convert 0xC3CC0000 as a V1 float to decimal
  - Sign: 1 -> Negative
  - Exponent: 0b1000 0111 → 135-127 = 8
  - Mantissa: 0b1001 1000… → 0b1.001 1000 = $1+2^{-3}+2^{-4}$
  - $(1+2^{-3}+2^{-4})*2^{8} = 2^{8}+2^{5}+2^{4}=304$
- Convert 0x00000000 as a V1 float to decimal
  - Sign: 0 -> Positive
  - Exponent: 0b0000 0000 → 0-127 = -127
  - Mantissa: 0b0000 0000 0000 0000 0000 000 → 0
  - $0*2^{-127}=0$

Agenda

- Endianness
- Function Pointers
- Void Pointers
- **Floating Point**
  - Simplified Model
  - **Optimizations**
  - IEEE-754 Standard

Optimization 1: The implicit 1

- Note that our mantissa is guaranteed to not have any leading zeros
  - If we wanted to write $0.234*10^5$, we'd instead write it as $2.340*10^4$
- In binary, every digit is only either 1 or 0. Since the MSB can't be 0, it must therefore be 1.
  - If the first bit will always be 1, we don't need to store it!
- Therefore, we can save 1 bit (or alternatively add another bit of precision) to the mantissa by not including the MSB of our mantissa
  - This is known as the implicit 1, and the resulting mantissa is a "normalized" number.

Floating Point System: V2

- A floating point number uses x bits for the exponent, and y bits for the mantissa (with x,y specified as parameters). For this slide, we'll use as an example a system with 3 exponent bits (bias of -3) and 4 mantissa bits

S XXX MMMM

- This represents the number $(-1)^{S} * 0b1.MMMM * 2^{(0bXXX+(-3))}$

Floating Point V2: Example

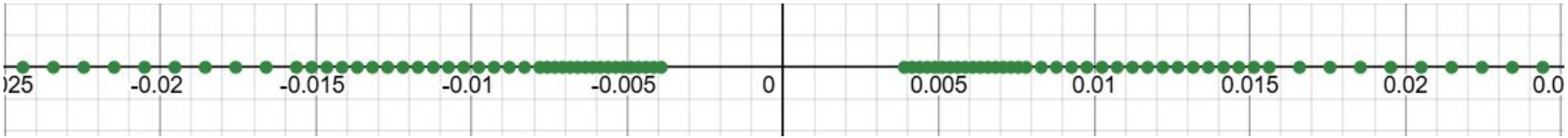
- For this example, let's assume we're working with a system with 3 exponent bits (bias of -3) and 4 mantissa bits.
- Convert 2.75 to a V1 float
- Step 1: Write the number in binary
  - 0b10.11 = 0b1.011000… * 2^1
- Step 2: Determine Sign/Exponent/Mantissa
  - Sign: Positive → 0
  - Exponent: 1-(-3) = 4 → 0b100
  - Mantissa: 0b0110
- Step 3: Concatenate
  - 0b0 100 0110
  - 0x46

Floating Point V2: Example

- For this example, let's assume we're working with a system with 8 exponent bits (bias of -127) and 23 mantissa bits.
- Convert 0x00000000 as a V2 float to decimal
  - Sign: 0 -> Positive
  - Exponent: 0b0000 0000 -> 0-127 = -127
  - Mantissa: 0b0000… → 0b1.000…
  - $1*2^{-127}$
- Convert 0x00000001 as a V2 float to decimal
  - Sign: 0 → Positive
  - Exponent: 0b0000 0000 → 0-127 = -127
  - Mantissa: 0b0000…1 → 0b1.000…1 = $1+2^{-23}$
  - $(1+2^{-23})*2^{-127}=2^{-127}+2^{-150}$

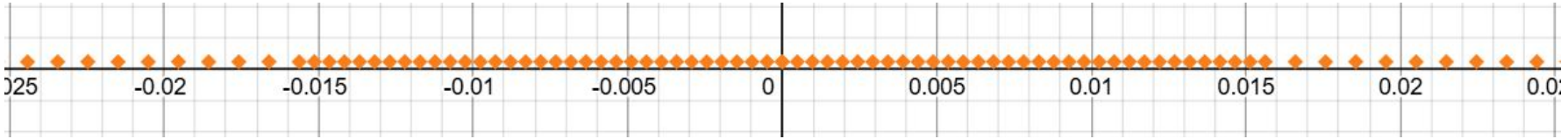
Problems with the implicit 1: Underflow

- The smallest number (in absolute value) we can represent is 2^{-127}
  - We can't represent 0
- The second smallest number is $2^{-127}+2^{-150}$
  - There's a big gap in the set of representable numbers (10 million times bigger than the gaps between consecutive numbers)
- This is known as an **underflow**; the result of computation gets too small to be represented.
- In the below diagram, each dot is a representable number (3 exponent, 4 mantissa bits).



Solution: Denormalized numbers

- Solution: Change the behavior of exponent 0b000…0 so that its range extends to 0
- How to do that?
  - If we use the same step size as exponent 0b000…1, then we can get exactly to 0
  - When dealing with these numbers, use an implicit 0 instead of an implicit 1, so it doesn't overlap with exponent 0b000…1
- Ends up losing precision at small numbers (so there's still underflow), but at least it's not a sudden cliff drop.



Floating Point System: V3

- A floating point number uses x bits for the exponent, and y bits for the mantissa (with x,y specified as parameters). For this slide, we'll use as an example a system with 3 exponent bits (bias of -3) and 4 mantissa bits

S XXX MMMM

- If the exponent bits are nonzero, then it represents the number $(-1)^S * 0b1.MMMM * 2^{(0bXXX+(-3))}$
- If the exponent bits are all zero, then it instead represents $(-1)^S * 0b0.MMMM * 2^{(0b000+(-3)+1)}$ (also known as a denormalized number, or "denorm")

Floating Point V3: Example

- For this example, let's assume we're working with a system with 8 exponent bits (bias of -127) and 23 mantissa bits.
- Convert 0x00000000 as a V2 float to decimal
  - Sign: 0 -> Positive
  - Exponent: 0b0000 0000. All zeros, so exponent meaning is 0-127+1 = -126
  - Mantissa: 0b0000… -> 0b0.000…
  - $0*2^{-126} = 0$ (We can represent 0!)
- Convert 0x00000001 as a V2 float to decimal
  - Sign: 0 -> Positive
  - Exponent: 0b0000 0000 -> 0-127+1 = -126
  - Mantissa: 0b0000…1 -> 0b0.000…1 = $0+2^{-23}$
  - $(0+2^{-23})*2^{-126}=2^{-149}$ (Much closer to 0)

Optimization 2: Dealing with infinity, division by zero

- We get a fairly large range using this (10^{38} with 8 exponent bits), but we can still exceed the maximum float.
- We also should deal with “1 / 0”
- Ideally, we’d like to include an “infinity” value to handle these cases
- While we’re at it, let’s add some values for error handling, that don’t correspond to any actual number
- What bitstrings do we use for this?
    - Since we want an infinity, let’s use the largest exponent for this

Floating Point System: Final Version

- An IEEE-754 standard binary floating point number uses x bits for the exponent, and y bits for the mantissa (with x,y specified as parameters). For this slide, we'll use as an example a system with 3 exponent bits (bias of -3) and 4 mantissa bits

S XXX MMMM

- If the exponent bits are nonzero and not all ones, then it represents the number $(-1)^S$*0b1.MMMM * $2^{(0bXXX+(-3))}$
- If the exponent bits are all zero, then it instead represents $(-1)^S$*0b0.MMMM * $2^{(0b000+(-3)+1)}$
- If the exponent bits are all ones, then:
  - If the mantissa is all zeros, it's either ∞ or -∞ (depending on the sign bit)
  - If the mantissa isn't all zeros, then it's a NaN (which means "Not a Number")

Floating Point System: Final Version

- An IEEE-754 standard binary floating point number uses x bits for the exponent, and y bits for the mantissa (with x,y specified as parameters). For this slide, we'll use as an example a system with 8 exponent bits (bias of -127) and 23 mantissa bits

SXXX XXXX XMMM MMMM MMMM MMMM MMMM MMMM

- If the exponent bits are nonzero and not all ones, then it represents the number $(-1)^S$ * 0b1.MMMM… * $2^{(0bXXXX\ XXXX+(-127))}$
- If the exponent bits are all zero, then it instead represents $(-1)^S$*0b0.MMMM… * $2^{(0b0+(-127)+1)}$
- If the exponent bits are all ones, then:
  - If the mantissa is all zeros, it's either ∞ or -∞ (depending on the sign bit)
  - If the mantissa isn't all zeros, then it's a NaN (which means "Not a Number")

Floating Point Final: Example

- For this example, let's assume we're working with a system with 8 exponent bits (bias of -127) and 23 mantissa bits.
- Convert 0xFF80 0000 as an IEEE-754 float to decimal
  - Sign: 1 -> Negative
  - Exponent: 0b1111 1111. All ones, so we're dealing with a special case
  - Mantissa: 0b0000… -> All zeros
  - -∞
- Convert 0xFF80 0001 as an IEEE-754 float to decimal
  - Sign: 1 -> Negative
  - Exponent: 0b1111 1111. All ones, so we're dealing with a special case
  - Mantissa: 0b0000…1 -> Not all zeros
  - NaN

Agenda

- Endianness
- Function Pointers
- Void Pointers
- **Floating Point**
  - Simplified Model
  - Optimizations
  - **IEEE-754 Standard**

IEEE-754: More notes on IEEE-754

- The IEEE-754 standard specifies five aspects:
- Arithmetic formats
  - The binary format, as discussed here. NaNs are supposed to use their mantissa to help with debugging purposes, but officially, you only need two types of NaNs (for quiet failures and signalling failures, respectively)
  - A decimal format which uses similar rules, but with decimal floating point instead. Due to the difficulty of writing decimal numbers in a binary encoding, the storage system of this format is *significantly* more complicated, and out of scope.
- Interchange formats

IEEE-754: Interchange Formats

- IEEE-754 specifies certain combinations of exponent and significand bits with special names:
- Single precision (also known as float):
  - 8 exponent bits, -127 bias, 23 mantissa bits
- Double precision (also known as double):
  - 11 exponent bits, -1023 bias, 52 mantissa bits
- Quad precision:
  - 15 exponent bits, 112 mantissa bits
- Octuple precision:
  - 19 exponent bits, 236 mantissa bits
- Half precision:
  - 5 exponent bits, 10 mantissa bits

IEEE-754: Interchange Formats

- C's float and double correspond to single-precision and double-precision floating point numbers, respectively
- Float:
  - Has enough precision for 6-7 decimal digits
  - Max value around 10^{38}
- Double:
  - Has enough precision for ~13 decimal digits
  - Max value around 10^{308}
  - Despite its name, it is generally considered the "default" floating point representation
  - You should almost always use a double; the precision loss is way more costly than a few bytes

IEEE-754: More notes on IEEE-754

- The IEEE-754 standard specifies five aspects:
- Arithmetic formats
  - The binary format, as discussed here. NaNs are supposed to use their mantissa to help with debugging purposes, but officially, you only need two types of NaNs (for quiet failures and signalling failures, respectively)
  - A decimal format which uses similar rules, but with decimal floating point instead. Due to the difficulty of writing decimal numbers in a binary encoding, the storage system of this format is *significantly* more complicated, and out of scope.
- Interchange formats
- Rounding Rules
- Operations
- Exception Handling (See skipped slides)

IEEE-754: Rounding Rules

- Several different options, but the most common one is “round to the nearest value, and break ties to the even number (last bit 0)
  - Ex. To round 14.5 to the nearest 2, go to 14, since 14 is closer to 14.5 than 16.
  - Ex. To round 15 to the nearest 2, go to 16, since 16 ends in more 0 bits than 14
- This is intended to be deterministic, but in practice, it’s rather unpredictable
- Generally a good idea not to rely too much on rounding behavior.
- Rounding happens after every operator in a string of operations, so we get slightly less precise results every step
  - This is why we use so many bits for the mantissa in doubles, and why doubles are the standard float option; precision is more important than range
  - Often different orders of doing operations yields different results even if they should be mathematically equivalent.

IEEE-754: More notes on IEEE-754

- The IEEE-754 standard specifies five aspects:
- Arithmetic formats
  - The binary format, as discussed here. NaNs are supposed to use their mantissa to help with debugging purposes, but officially, you only need two types of NaNs (for quiet failures and signalling failures, respectively)
  - A decimal format which uses similar rules, but with decimal floating point instead. Due to the difficulty of writing decimal numbers in a binary encoding, the storage system of this format is *significantly* more complicated, and out of scope.
- Interchange formats
- Rounding Rules
- Operations

IEEE-754: Operations

- Requires conversion to/from integer, arithmetic, comparisons, abs/negate, floor/ceiling, etc.
  - Two new operators are: square root and fused multiply-add (a*b+c done in one step)
- Additionally recommends native support for other math operations (e^x, 2^x, 10^x, log x, distance formula, trig functions, hypertrig functions, arctrig functions)
- Most of these functions are available through the <math.h> library

IEEE-754: More notes on IEEE-754

- The IEEE-754 standard specifies five aspects:
- Arithmetic formats
  - The binary format, as discussed here. NaNs are supposed to use their mantissa to help with debugging purposes, but officially, you only need two types of NaNs (for quiet failures and signalling failures, respectively)
  - A decimal format which uses similar rules, but with decimal floating point instead. Due to the difficulty of writing decimal numbers in a binary encoding, the storage system of this format is *significantly* more complicated, and out of scope.
- Interchange formats
- Rounding Rules
- Operations
- Exception Handling

IEEE-754: Exception Handling

- Defines five classes of exceptions, along with recommended behavior:
- Invalid operation
  - Ex. sqrt(-1.0)
  - By default, returns a NaN (quiet)
- Division by zero
  - By default, returns infinity
- Overflow
  - By default, returns infinity
- Underflow
  - Returns a denorm. Note that we consider any result using a denorm to suffer from underflow (due to precision loss)
- Inexact value
  - Any math that yields a number that can’t be exactly represented, like 1.0/3
  - Rounds to a representable number according to the rounding rule.